

Modélisation

Étape A : Préparation avant modélisation

Encodage → Split stratifié → Imputation (fit sur train, transform sur test)

- **Pourquoi ?**
 - L'encodage avant le split est acceptable car il s'agit d'une transformation déterministe : il ne « apprend » rien du dataset, il traduit simplement des modalités connues a priori. Encoder après le split ne changerait rien au résultat, mais le faire avant simplifie la gestion des colonnes.
 - L'imputation après le split est obligatoire car elle apprend quelque chose (moyenne, médiane, mode). Si l'on impute sur l'ensemble complet avant de splitter, les statistiques calculées sur le test contaminent l'entraînement — c'est du *data leakage*.

1) L'encodage

L'encodage est une étape importante car il permet de transformer des variables catégorielles en valeurs numériques utilisables par les modèles.

- **One-Hot Encoding (OHE)** : utilisé pour les variables sans ordre naturel (*genre_client, plan_tarifaire, moyen_paiement, type_abonnement, zone_reseau_principale*).
 - Ces variables n'ont pas d'ordre défendable. Assigner des entiers induirait artificiellement une relation ordinale. OHE crée une colonne binaire par modalité, sans imposer d'ordre.
- **Encodage ordinal** : utilisé pour *qualite_signal_dominante*.
 - L'ordre est réel et mesurable (*Faible < Bon < Excellent*). OHE détruirait cette information, l'encodage ordinal la préserve.

2) Le split stratifié

Le split consiste à découper le dataset en deux parties : une pour l'entraînement et une autre pour tester les performances.

Avec 300 observations et 27,7 % de churn (83 positifs, 217 négatifs), un split aléatoire risquerait de déséquilibrer le ratio churn/non-churn. Le split stratifié

garantit que les deux sous-ensembles reproduisent exactement ce ratio.

→ 70 % du dataset est dédié à l'entraînement et 30 % au test.

3) L'imputation

L'imputation consiste à remplacer les valeurs manquantes par des valeurs calculées ou définies. Nous avons 8 colonnes avec des NaN de natures différentes. La stratégie dépend du mécanisme de génération des NaN (MAR, MCAR, MNAR).

Variable	Type	Stratégie d'imputation	Justification
facture_moyenne_mensuelle	Continue	Médiane par groupe (type_abonnement)	Les factures dépendent du type d'abonnement ; la médiane est robuste aux outliers
satisfaction_client	Discrète (1-5)	Médiane globale	Variable subjective sans groupe naturel fort
data_manquante	Binaire	Mode (0 ou 1)	Seul le mode a un sens
reclamation_manquante	Binaire	Mode (0 ou 1)	Idem
tendance_data_pct	Continue, ratio	Médiane globale	Ratio asymétrique ; médiane robuste
ratio_data_voix	Continue, ratio	Médiane globale	Présence de zéros structurels
ratio_sms_appels	Continue, ratio	Médiane globale	Idem
score_frustration	Continue, composite	0 (valeur neutre)	NaN = absence de réclamation et satisfaction non renseignée → 0 est la valeur métier correcte

👉 Le *score_frustration* est le seul cas où l'imputation par 0 est justifiée métier et non statistique.

-

Étape B : Modélisation

Face au problème de churn, plusieurs algorithmes de classification supervisée sont envisageables. Les modèles retenus sont :

- **XGBoost (baseline)** : point de départ, gère nativement les déséquilibres via *scale_pos_weight*, tolère les échelles différentes, produit des SHAP values.
- **Random Forest** : sert de référence pour vérifier l'apport du boosting.
- **Régression Logistique** : modèle linéaire, simple et interprétable.

1) Gestion du déséquilibre

Dataset : 72,3 % non-churn / 27,7 % churn. Ce déséquilibre peut biaiser le modèle vers la classe majoritaire.

Deux stratégies sont testées :

- **Class_weight natif** : pénaliser davantage les erreurs sur la classe minoritaire. Implémentation : *class_weight='balanced'* (LR, RF) ou *scale_pos_weight = 217/83 ≈ 2,61* (XGBoost).
- **SMOTE (Synthetic Minority Over-sampling Technique)** : génère des observations synthétiques de churners pour rééquilibrer le train set à 50/50. Attention : SMOTE s'applique uniquement sur *X_train*, jamais sur *X_test*.

2) Évaluation des performances

Les modèles seront évalués selon trois métriques prioritaires :

- **AUC-ROC (priorité 1)** : capacité à distinguer churner / non-churner indépendamment du seuil.
- **F1-score (priorité 2)** : moyenne harmonique de la précision et du rappel sur la classe churn.
- **Recall / Rappel (priorité 3)** : proportion de churners correctement détectés.